

8. Header Linked List

–Delete At End

Program:

```
void deleteFromEnd()
{
    if (head->next == nullptr)
    {
        cout << "List is empty. Nothing to delete." << endl;
        return;
    }

    Node *temp = head;

    // Traverse to the node BEFORE the last node
    while (temp->next->next != nullptr)
    {
        temp = temp->next;
    }

    // temp->next is the node to be deleted
    Node *nodeToDelete = temp->next;

    // Unlink
    temp->next = nullptr;

    cout << "Deleted " << nodeToDelete->data << " from the
end." << endl;

    free(nodeToDelete);
    nodeToDelete = nullptr;

    // Decrement internal count
    head->data--;
}
...•
```

```

case 6:
    deleteFromEnd();
    break;

```

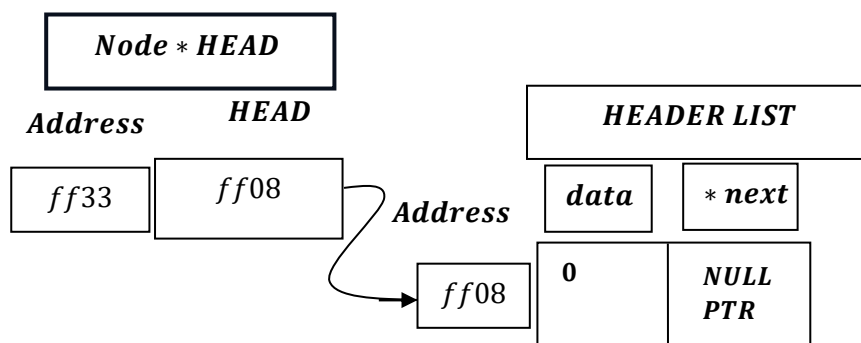
Program Analysis

```

if (head->next == nullptr)
{
    cout << "List is empty. Nothing to delete." << endl;
    return;
}

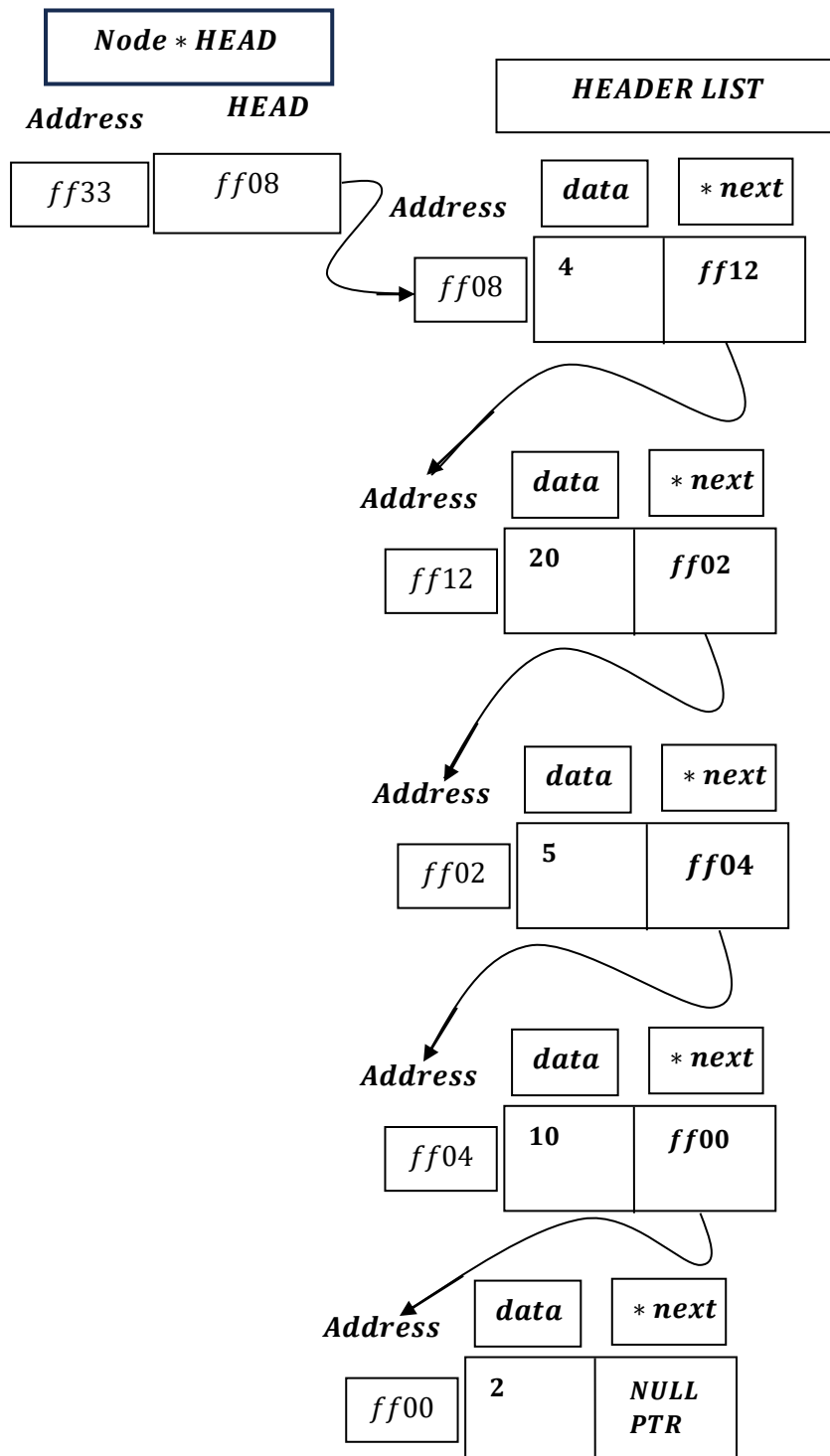
```

i.e.

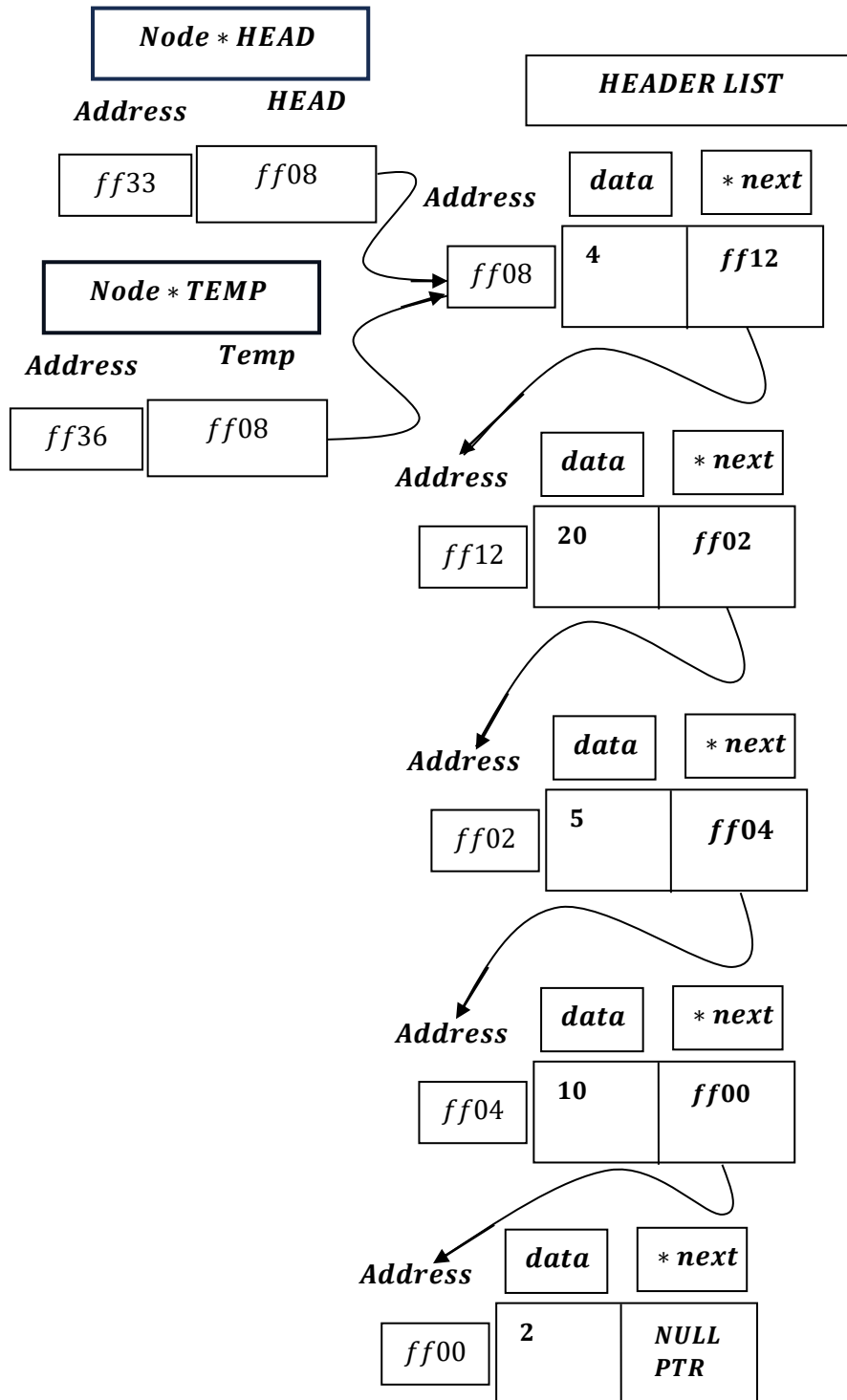


head → *next* = *NULLPTR* , *head* → *Data* = *0* , then list is empty .

If List is not Empty



```
Node *temp = head;  
Temp = Head = ff08.
```



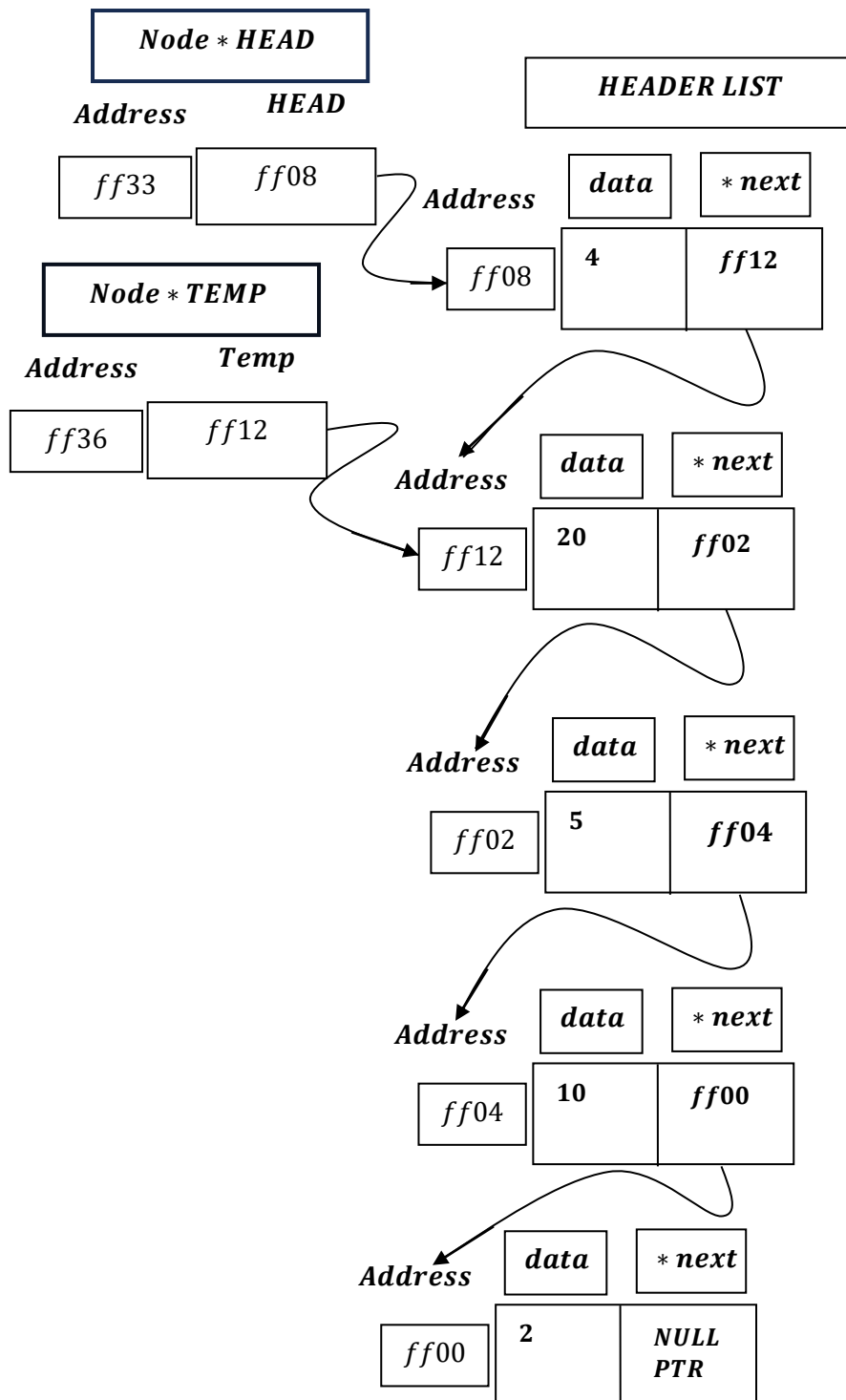
```

while (temp->next->next != nullptr)
{
    temp = temp->next;
}

```

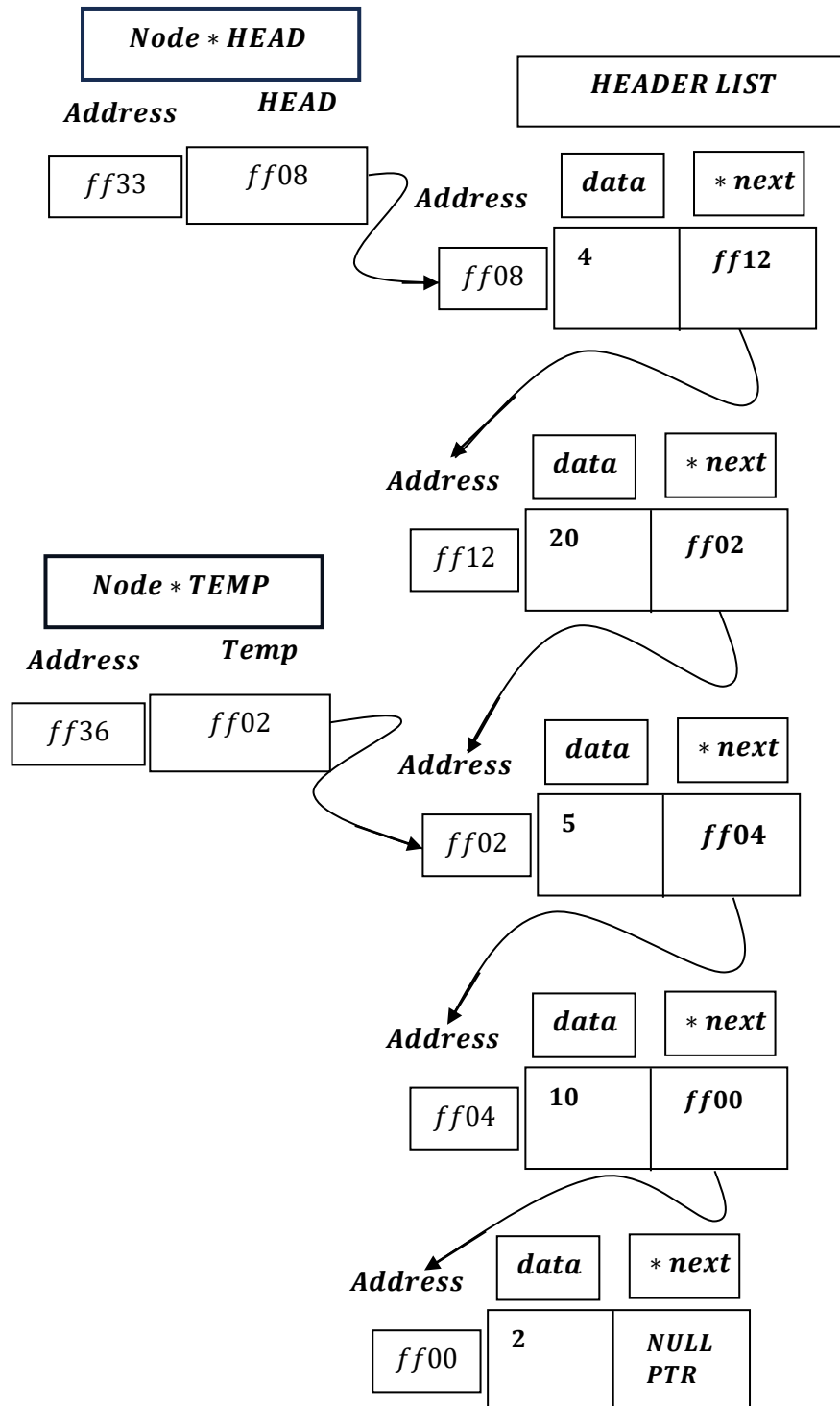
TEMP → NEXT → NEXT = ff02 ≠ NULLPTR is TRUE:

TEMP = TEMP → NEXT = ff12



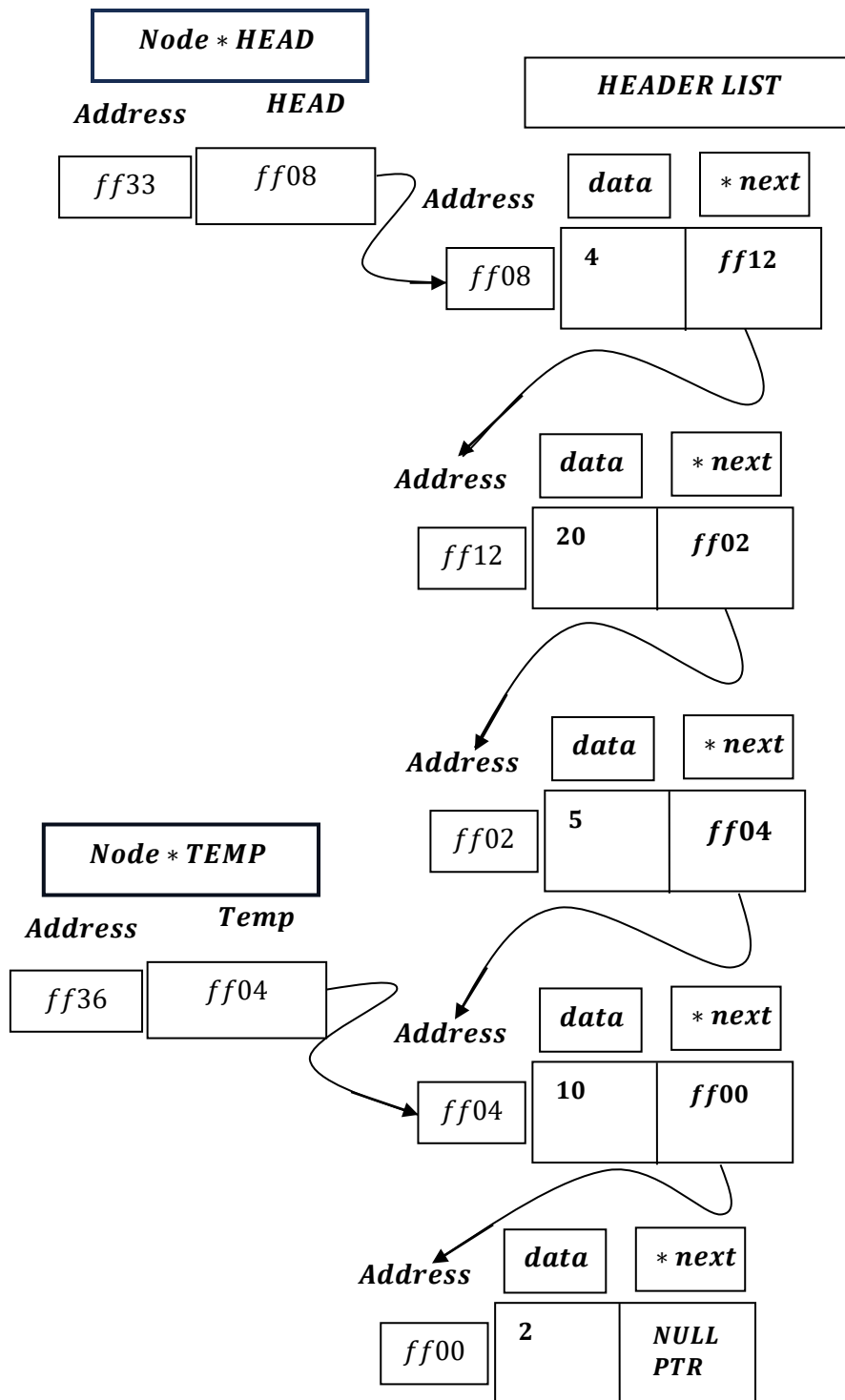
$TEMP \rightarrow NEXT \rightarrow NEXT = ff04 \neq NULLPTR$ is TRUE:

$TEMP = TEMP \rightarrow NEXT = ff02$



$TEMP \rightarrow NEXT \rightarrow NEXT = ff00 \neq NULLPTR$ is TRUE:

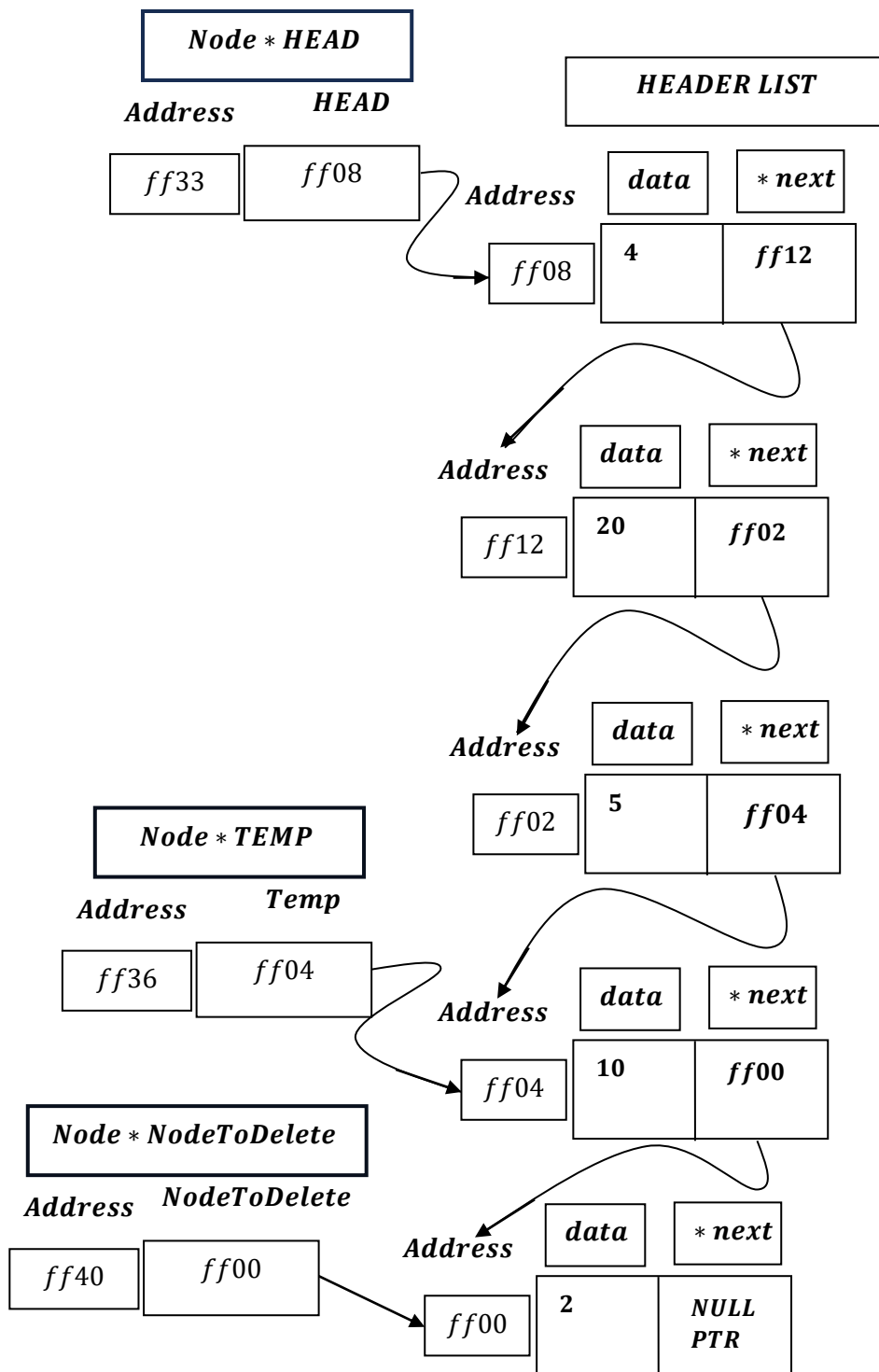
$TEMP = TEMP \rightarrow NEXT = ff04$



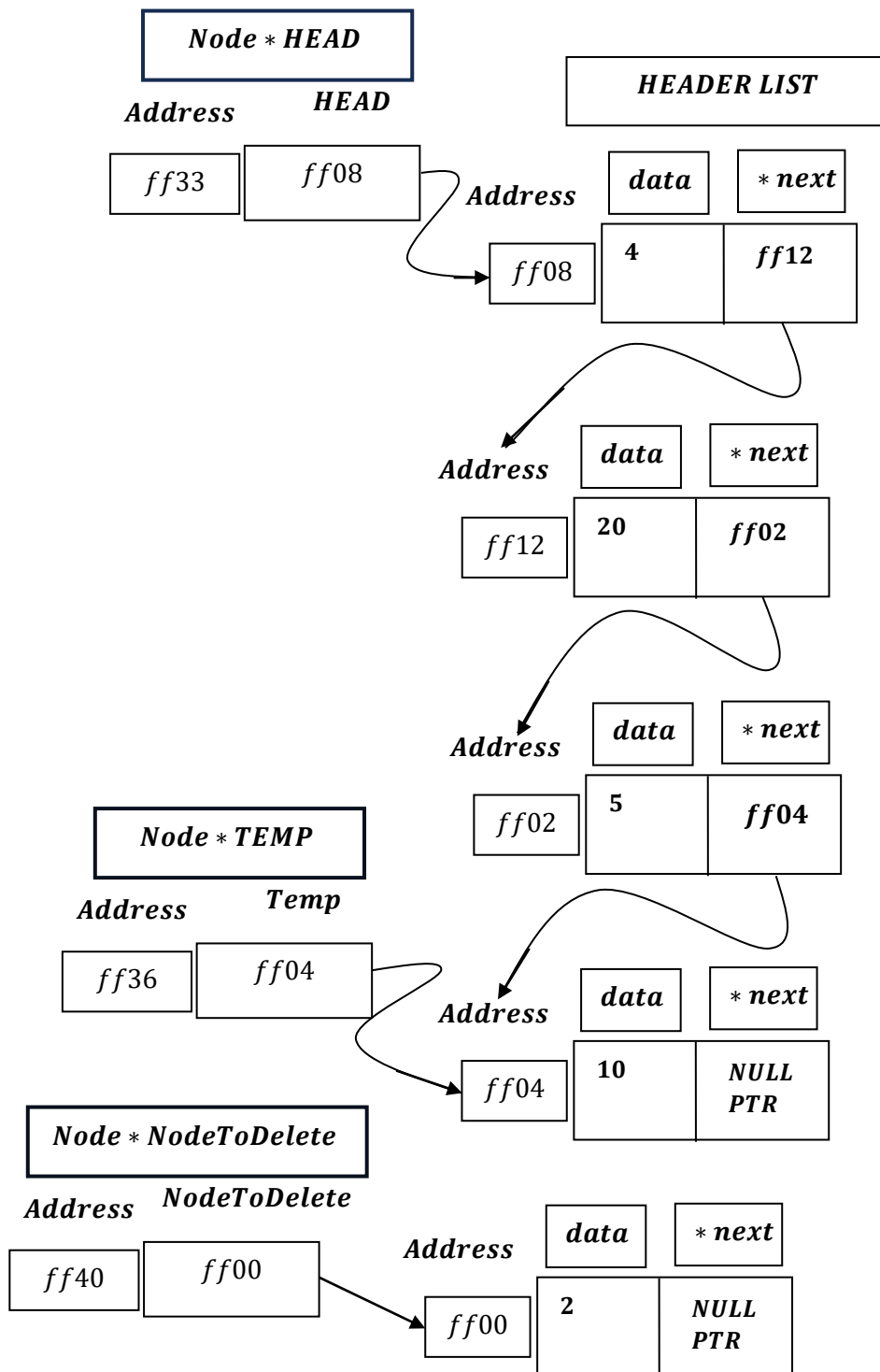
$TEMP \rightarrow NEXT \rightarrow NEXT = NULLPTR \neq NULLPTR$ is FALSE, hence, exit from loop.

```
Node *nodeToDelete = temp->next;
```

$NodeToDelete = TEMP \rightarrow NEXT = ff00$.



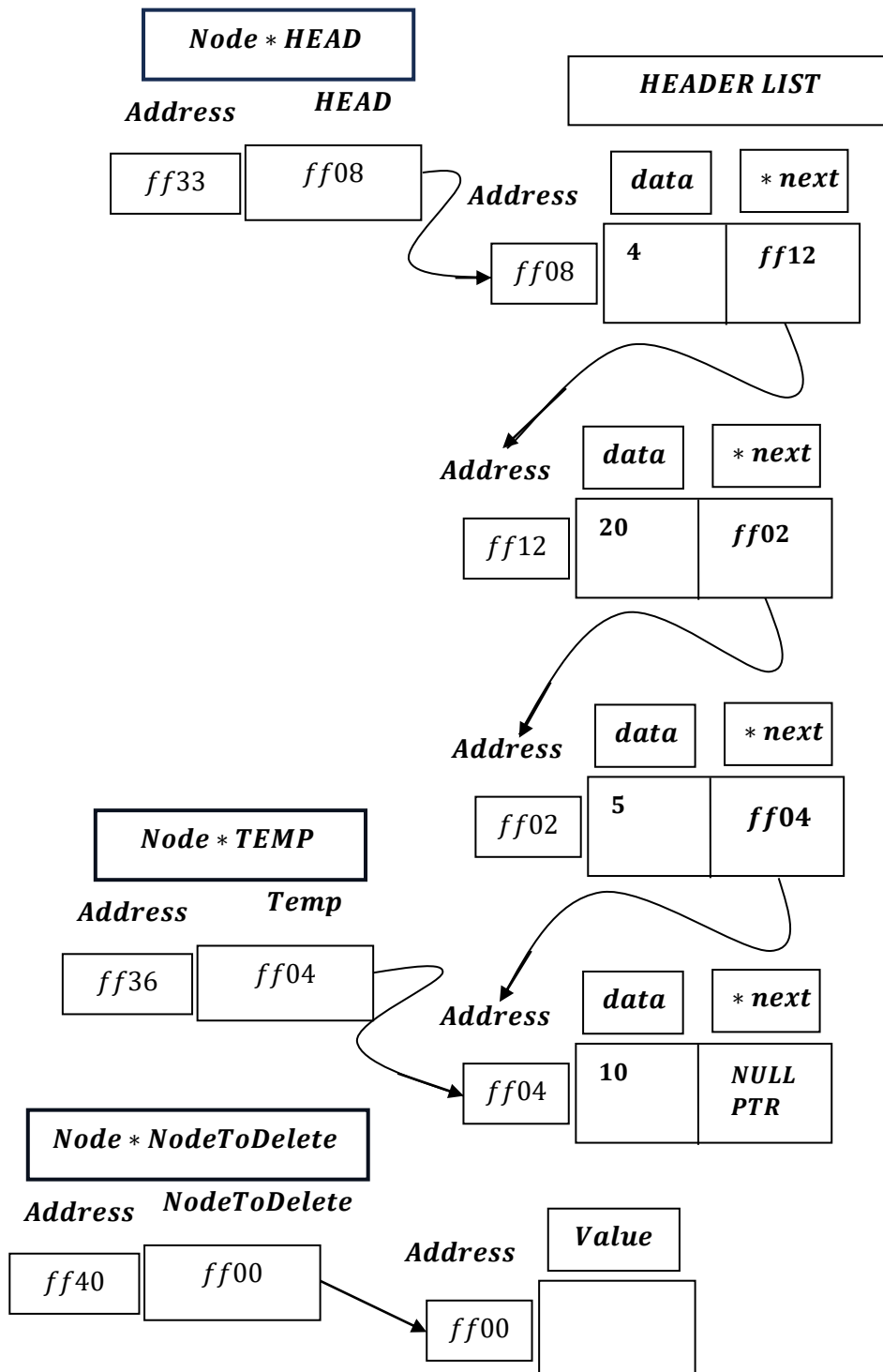

```
temp->next = nullptr;  
TEMP → NEXT = NULLPTR.
```



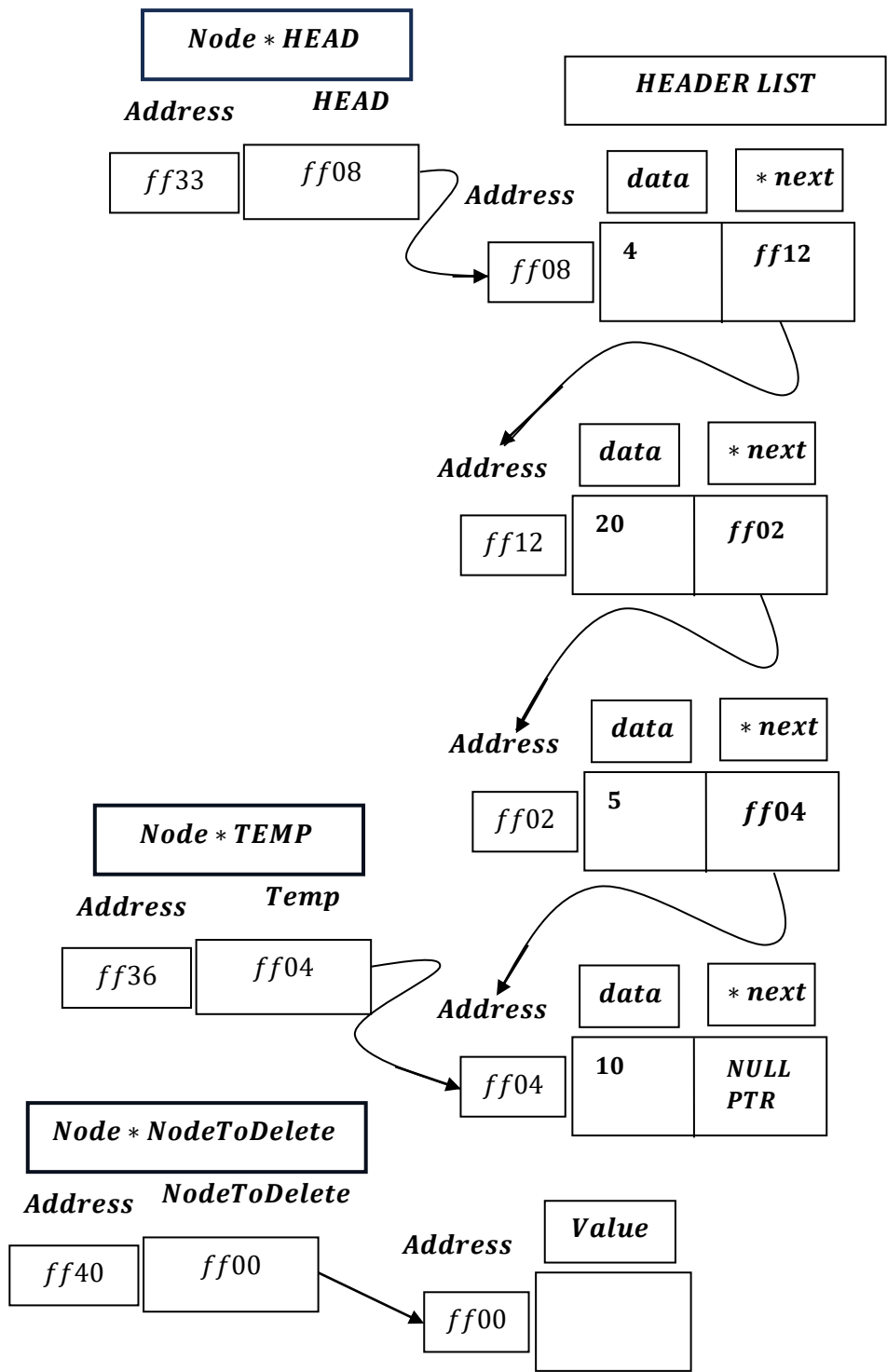
```
cout << "Deleted " << nodeToDelete->data << " from the end." << endl;
```

Output $\Rightarrow$ Print: Deleted `2` from the end.

```
free(nodeToDelete);
```

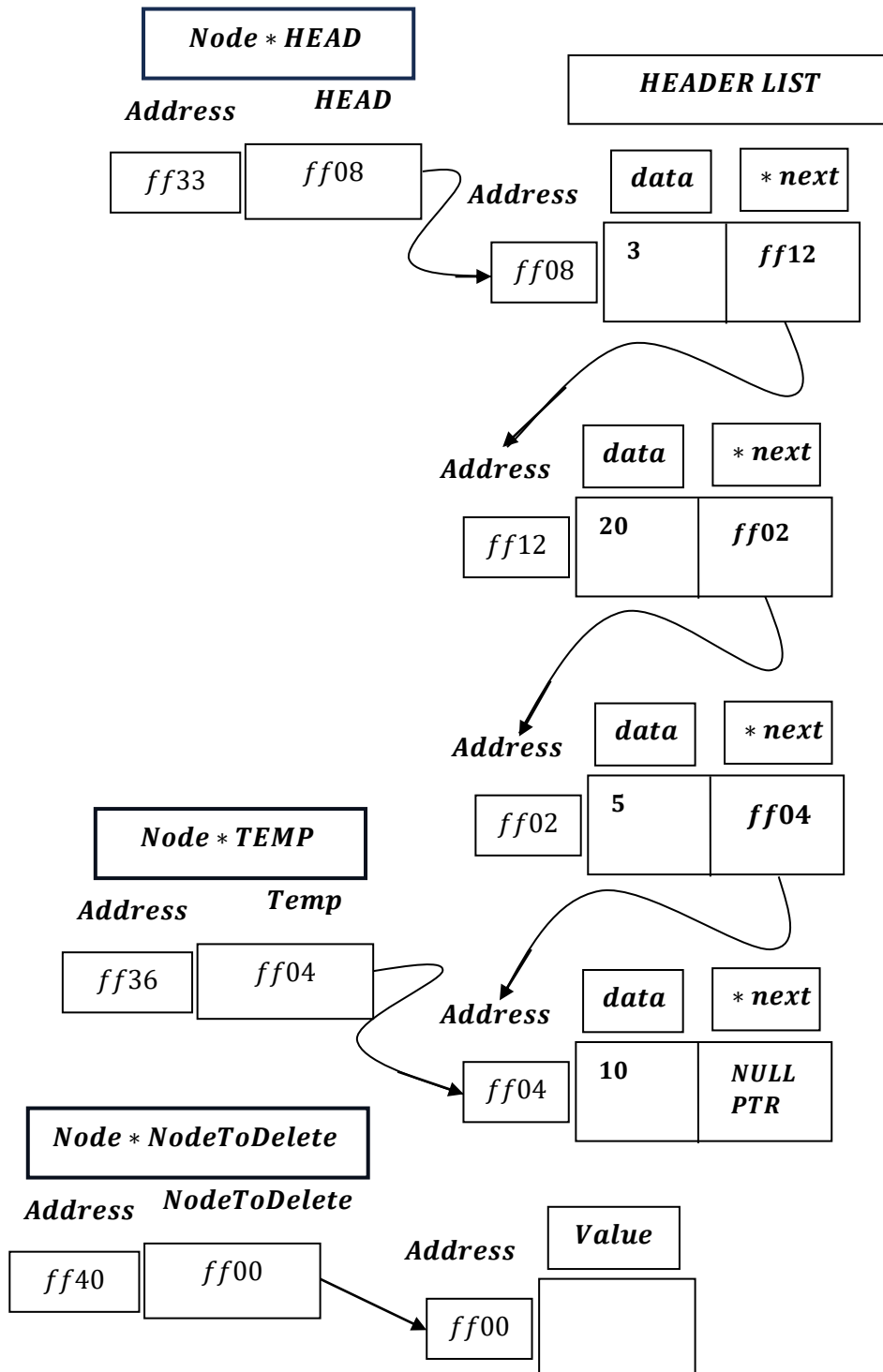


```
nodeToDelete = nullptr;
```



`head->data--;`

$Head \rightarrow Data -- \Rightarrow Head \rightarrow Data = Head \rightarrow Data - 1 = 4 - 1 = 3.$



And we get our list:

